

Простое и практичное объяснение ключевых тем по Haskell для лабораторных

Executive summary

Этот отчёт группирует ваши темы в практичные «узлы», которые чаще всего встречаются в лабораторных: (1) управление потоком вычислений и сопоставление с образцом, (2) типы и собственные структуры данных, (3) безопасная работа с ошибками и разбором строк, (4) обходы структур (map/traverse) и свёртки (foldl/foldr) с учётом ленивости, (5) операторы и «склейка» маленьких функций, (6) практические приёмы: GHCi, отладка, QuickCheck и типичные ошибки.

1

Ключевая идея: в Haskell «управление потоком» часто делается не отдельными операторами цикла/ветвления, а **через типы** (Maybe/Either), **сопоставление с образцом** (pattern matching) и **композицию** функций (.) и эффектов (traverse/mapM). 2

Неуточнённые термины из запроса и принятая интерпретация: - **ReadMaybe**: чаще всего студенты имеют в виду `readMaybe` из `Text.Read` (безопасный парсер, возвращает `Maybe`). В отчёте именно так и используется. 3

- **die**: в стандартной библиотеке (base) есть `System.Exit.die :: String -> IO a`, который печатает сообщение в `stderr` и завершает программу с `exitFailure`. 4

Авторитетные источники, на которые опирается отчёт: Haskell 2010 Language Report 5, документация base/mtl/transformers, а также книги Learn You a Haskell for Great Good! 6 (автор Миран Липовача 7) и Haskell Programming from First Principles 8 (авторы Chris Allen 9 и Julie Moronuki 10). 11

Управление потоком и синтаксис

Условие `if ... then ... else ...`

Когда использовать.

`if` подходит, когда у вас **есть булево условие** (`Bool`) и нужно выбрать одно из двух выражений. В Haskell это **выражение**, а не оператор: оно возвращает значение. По стандарту `if e1 then e2 else e3` возвращает `e2`, если `e1` вычисляется в `True`, `e3` — если в `False`, и даёт $\perp$ (не завершается/падает), если `e1` не вычисляется до значения (например, $\perp$).

12

Важно: в Haskell `if` всегда требует `else`. Это «дисциплинирует» код: у выражения должен быть результат в любом случае. 12

Факт, который помогает запомнить: `if` — это частный случай `case` по `Bool`. Стандарт прямо даёт тождество трансляции:

`if e1 then e2 else e3 = case e1 of { True -> e2 ; False -> e3 }.` 12

Короткий рабочий пример:

```
absInt :: Int -> Int
absInt x = if x < 0 then -x else x
```

- Пояснение: 1) `x < 0` вычисляется в `Bool`.
2) Если `True` — берётся `-x`, иначе — `x`.
3) Возвращается `Int`, поэтому обе ветки тоже `Int`. ¹²

`case ... of` и сопоставление с образцом

Когда использовать `case`.

`case` нужен, когда вы хотите **разобрать значение по форме** (конструктору/структуре) и выбрать вариант выполнения: список пустой или нет, `Maybe` это `Nothing` или `Just`, ваше дерево — пустое или вершина, и т.д. ¹³

По стандарту: альтернативы `case` пробуются **сверху вниз**, и каждая альтернатива может иметь **guards** (охраняющие условия). ¹⁴

Классический пример со списком:

```
describeList :: [a] -> String
describeList xs =
  case xs of
    []      -> "пустой список"
    [_]     -> "ровно один элемент"
    (_:_:_) -> "два или больше"
```

- Пояснение: 1) `case xs of ...` сравнивает `xs` с образцами.
2) `[]` матчится только с пустым списком.
3) `[_]` матчится со списком из одного элемента (значение игнорируем).
4) `(_:_:_)` говорит «есть как минимум два элемента». ¹³

Pattern matching в определениях функций vs `case`

Почему **pattern matching** в аргументах функций часто лучше читается.

Вместо `case` можно писать несколько уравнений функции — это тот же принцип сопоставления, но синтаксис обычно короче. Стандарт описывает, что семантика таких конструкций сводится к `case`-разбору кортежа аргументов. ¹⁵

Пример:

```
len :: [a] -> Int
len []      = 0
len (_:xs) = 1 + len xs
```

Пояснение: 1) Первый случай: пустой список $\rightarrow 0$.

2) Второй: голова есть, хвост `xs` $\rightarrow 1 +$ длина хвоста.

3) Рекурсия по структуре списка — типичная замена «циклам» в императивных языках. ¹²

Guards и as-patterns

Guards (охранные условия) удобны, когда разбор по конструктору недостаточен и нужно добавить условия. Стандарт различает: - pattern guards (`p <- e`), - локальные объявления `let decls`, - булевы guards (`Bool`-выражения). ¹⁴

`as-pattern` (`var@pat`) позволяет одновременно: - получить доступ к частям значения через `pat`, - сохранить всё значение целиком под именем `var`. ¹²

Пример (и guards, и as-pattern):

```
dropLeadingZero :: [Int] -> [Int]
dropLeadingZero xs@(x:rest)
  | x == 0    = rest
  | otherwise = xs
dropLeadingZero [] = []
```

Пояснение: 1) `xs@(x:rest)` матчится только с непустым списком.

2) `x` — первый элемент, `rest` — хвост, а `xs` — исходный список целиком.

3) Если `x == 0`, возвращаем `rest`, иначе — исходный `xs`. ¹⁶

do -нотация

Когда и зачем использовать do.

`do` — это «удобная запись» для **последовательного связывания вычислений в монаде** (IO, Maybe, State и т.д.). Стандарт прямо показывает: `do` транслируется в цепочки `(>>)` и `(>>=)`; а строка вида `p <- e` разворачивается через `(>>=)` и обработку неудачного сопоставления через `fail`. ¹⁷

Пример do с IO

```
main :: IO ()
main = do
  putStrLn "Введите строку:"
  line <- getLine
  let ws = words line
  print ws
```

Пояснение шаг за шагом: 1) `putStrLn` печатает приглашение. (В Prelude `putStrLn` определён через `do`: сначала `putStr`, потом перевод строки.) ¹⁸

2) `line <- getLine` выполняет IO-действие чтения строки и связывает результат с именем `line`. (В Prelude `getLine` тоже определён через `do` и `if`.) ¹⁹

3) `let ws = words line` — локальная привязка без IO: просто вычисление списка слов. ²⁰

4) `print ws` печатает значение, используя `show`, и добавляет перевод строки. (В Prelude `print x = putStrLn (show x)`.) ¹⁸

Пример `do` с `Maybe`

`Maybe` — «контейнер возможного значения»: либо `Just a`, либо `Nothing`. Он удобен для «раннего выхода» без исключений. ²¹

```
import Text.Read (readMaybe)

sumTwo :: String -> Maybe Int
sumTwo s = do
  case words s of
    [aStr, bStr] -> do
      a <- readMaybe aStr
      b <- readMaybe bStr
      pure (a + b)
    _ -> Nothing
```

Пояснение: 1) `words s` режет строку на токены по пробельным символам. ²²

2) Если токенов не ровно два — сразу `Nothing` (ошибка формата).

3) `readMaybe` безопасно парсит число: `Just n` при успехе, `Nothing` при неуспехе. ³

4) `a <- readMaybe aStr` и `b <- ...` «протаскивают» `Nothing`: если любой парсинг не удался, весь результат станет `Nothing`. Это соответствует смыслу `Maybe` как «простого error-монада». ²³

5) `pure (a + b)` поднимает значение в `Maybe`. Для монад ожидается закон `pure = return`. ²⁴

Пример `do` с монадой `State`

Состояние — типичный кейс лабораторных: «счётчик», «нумерация», «накопление промежуточных вычислений». `State` — это вычисление, которое неявно протаскивает состояние типа `s`. В документации подчёркнуто: `return` оставляет состояние неизменным, а `(>>=)` передаёт финальное состояние первого вычисления как начальное для второго. ²⁵

```
import Control.Monad.Trans.State.Strict (State, get, put, evalState)
import Data.Traversable (traverse)

fresh :: State Int Int
fresh = do
  n <- get
  put (n + 1)
  pure n

number :: [a] -> State Int [(Int, a)]
number xs =
  traverse (\x -> do
    n <- fresh
    pure (n, x)
```

```
) xs
```

```
example :: [(Int, String)]  
example = evalState (number ["a", "b", "c"]) 0
```

Пояснение: 1) `fresh` читает текущее состояние `n` (`get`) и обновляет его на `n+1` (`put`). ²⁶
2) `traverse` идёт слева направо, выполняет действие для каждого элемента и собирает результаты в той же форме структуры (здесь — список). ²⁷
3) Внутри `traverse` мы делаем `fresh`, получаем метку `n` и возвращаем пару `(n, x)`.
4) `evalState ... 0` запускает вычисление со стартовым состоянием 0 и возвращает только результат (финальное состояние отбрасывается). ²⁸

Типы и пользовательские структуры данных

Базовые типы и параметризация

Базовая практика: понимать, что многие «знакомые» штуки — **типы и конструкторы**: - `String` в Prelude — это **синоним** для `[Char]`. ²⁹
- Списки — это рекурсивный тип `[] | (:)`. ³⁰
- `Maybe a` имеет два конструктора `Nothing | Just a`. ³¹
- `Either a b` имеет `Left a | Right b`. ³²

Параметризованный тип (обобщённый) — это тип с переменной типа: `Maybe a`, `[a]`, `Either e a`. Идея: «одна форма контейнера» может хранить значения разных типов. ³³

`data`, `newtype`, `type`

`type` (**синоним типа**).

`type T a = ...` вводит новое имя, полностью эквивалентное старому типу после подстановки параметров; это «чисто синтаксический» механизм для читабельности. В отчёте подчёркивается: синоним и его определение взаимозаменяемы (с нюансами в `instance`). ³⁴

`newtype` (**переименование/новая «обёртка» без накладных расходов**).

`newtype` создаёт новый тип с тем же представлением, но **отдельной идентичностью в системе типов**; преобразование через конструктор может быть без runtime-накладных расходов. Также подчёркнуто отличие от `type`: `newtype` нужно явно «коэрцировать», и `newtype` может участвовать в рекурсивных типах. ³⁵

`data` (**алгебраический тип данных, ADT**).

`data` вводит новый тип-конструктор и набор конструкторов данных; может быть несколько конструкторов. Отметка строгого поля `!` влияет на то, когда аргументы конструктора вычисляются. ³⁵

Сравнение «на пальцах» (что чаще всего спрашивают в лабораторных):

```

type UserId1 = Int
newtype UserId2 = UserId2 Int
data UserId3 = UserId3 Int

```

Пояснение: - `UserId1` — просто другое имя для `Int`; компилятор не различает их «по смыслу».

35

- `UserId2` и `UserId3` — разные типы, и их нельзя случайно перепутать с `Int` без явного преобразования. 35

- `newtype` (в отличие от `data`) не добавляет «лишний уровень» представления, и важный нюанс: `N ⊥ = ⊥` для `newtype` (конструктор «unlifted»), а для `data` поведение может отличаться.

36

Алгебраические типы и деревья

ADT почти всегда воспринимается как «сумма вариантов» (конструкторы через `|`) + «произведение полей» (поля конструктора).

Визуально `Maybe` / `Either` и дерево можно держать в голове как «граф вариантов»:

```

graph TD
    MaybeA["Maybe a"] --> Nothing["Nothing"]
    MaybeA --> JustA["Just a"]

    EitherEA["Either e a"] --> LeftE["Left e"]
    EitherEA --> RightA["Right a"]

```

`Maybe` и `Either` как структуры данных подтверждаются стандартной документацией `base` и их определениями в `Prelude/Report`. 33

Пример: бинарное дерево поиска

```

data Tree a
  = Empty
  | Node (Tree a) a (Tree a)
  deriving (Show, Eq)

insert :: Ord a => a -> Tree a -> Tree a
insert x Empty = Node Empty x Empty
insert x (Node l v r)
  | x < v      = Node (insert x l) v r
  | x > v      = Node l v (insert x r)
  | otherwise = Node l v r

member :: Ord a => a -> Tree a -> Bool
member _ Empty = False
member x (Node l v r)
  | x < v      = member x l
  | x > v      = member x r

```

```

    | otherwise = True

inorder :: Tree a -> [a]
inorder Empty = []
inorder (Node l v r) = inorder l ++ [v] ++ inorder r

```

Пояснение: 1) Тип `Tree a` рекурсивный: `Node` содержит два поддерева и значение.

2) `insert` / `member` используют `guards` для сравнения `x` и `v`. `Guards` синтаксически поддерживаются в уравнениях и `case`. ³⁷

3) `inorder` — типичная рекурсия «слево → корень → право».

4) В `inorder` используется `++` (склейка списков). В `PreludeList` у `++` ассоциативность `infixr`

5, и определение рекурсивное: пустой список слева возвращает правый, непустой добавляет голову и рекурсивно присоединяет хвост. ³⁸

Диаграмма формы дерева:

```

graph TD
    T["Tree a"] --> E["Empty"]
    T --> N["Node (Tree a) a (Tree a)"]
    N --> L["левое поддерево"]
    N --> V["значение a"]
    N --> R["правое поддерево"]

```

Контейнеры, ошибки и безопасный разбор строк

`Maybe` и `Either` как «контейнеры ошибок»

`Maybe a`: «значение может отсутствовать». Документация `base` подчёркивает: `Maybe` — хороший способ обрабатывать ошибки/исключительные случаи без `error`, и `Maybe` — это монада «простого» типа, где все ошибки представлены `Nothing`. ³⁹

`Either e a`: «либо ошибка `e`, либо корректный результат `a`». По соглашению `Left` хранит ошибку, `Right` — корректное значение. ⁴⁰

Таблица «Just/Nothing vs Left/Right»:

Что хранит	Конструкторы	Типовая интерпретация в задачах	«Что делать дальше»
<code>Maybe a</code>	<code>Nothing</code> , <code>Just a</code>	Ошибка без причины / «значения нет»	либо <code>case</code> , либо <code>maybe default f</code>
<code>Either e a</code>	<code>Left e</code> , <code>Right a</code>	Ошибка с причиной	либо <code>case</code> , либо <code>either onErr onOk</code>

`maybe` и `either` как «функции разборки» задаются в `Prelude/Report` и/или в документации `Data.Either`. ⁴¹

Read, read, reads и «ReadMaybe»

Интерпретация вашего пункта «Read/ReadMaybe».

- `Read` — это typeclass «умеет парситься из String». ⁴²
- `reads` возвращает список возможных разборов (вместе с непрочитанным хвостом строки). В Prelude определён тип `ReadS a = String -> [(a,String)]`. ⁴³
- `read :: Read a => String -> a` **опасен**: падает с `error`, если парсинг не удался; документация `Text.Read` прямо говорит, что `read` “discouraged” для реальных программ и предлагает `readMaybe` / `readEither` как безопасные альтернативы. ³

В Prelude (Report) видно и почему: `read` пробует `reads`, и если вариантов 0 или больше 1, вызывает `error`. ¹⁸

`readMaybe` (то самое «ReadMaybe» в вашем вопросе) — безопасная версия:

```
readMaybe :: Read a => String -> Maybe a. 3
```

`readEither` — тоже безопасная, но даёт причину:

```
readEither :: Read a => String -> Either String a. 3
```

Практический пример: разбор списка чисел

Пусть пользователь вводит строку `"10 20 30"`.

Неправильная (в смысле результата) композиция: `map readMaybe` даёт «список попыток»

```
import Text.Read (readMaybe)

parseListBad :: String -> [Maybe Int]
parseListBad s = map readMaybe (words s)
```

Что получается: - тип `parseListBad` говорит: у вас столько же элементов, сколько токенов, но каждый может быть `Nothing`.

- часто в лабораторных нужен результат «либо всё успешно, либо ошибка». Для этого нужен `traverse` / `mapM`. ⁴⁴

Правильная для “всё или ничего”: `traverse readMaybe`

```
import Text.Read (readMaybe)

parseListMaybe :: String -> Maybe [Int]
parseListMaybe s = traverse readMaybe (words s)
```

Пояснение: 1) `words s :: [String]`. ²²

2) `readMaybe :: String -> Maybe Int`. ³

3) `traverse :: Applicative f => (a -> f b) -> t a -> f (t b)` запускает парсинг по всем элементам и собирает. Если встречается `Nothing`, результат «закорачивается» (short-circuit). ²⁷

Для `Either` то же самое, но с сообщением ошибки:

```
import Text.Read (readEither)

parseListEither :: String -> Either String [Int]
parseListEither s = traverse readEither (words s)
```

Здесь `Left "..."` позволит вам в лабораторной красиво вывести ошибку пользователю вместо падения программы. ⁴⁵

Функции высшего порядка, обходы и свёртки с учётом ленивости

`map`, `traverse`, `mapM`, `sequence` и когда что выбирать

Главное различие:

- `map` работает в «чистом мире»: применяет функцию к каждому элементу и не умеет «выполнять эффекты». ⁴⁶
- `traverse` умеет пробегать структуру и «выполнять эффекты» `Applicative/Monad` слева направо, собирая результаты. ²⁷
- `mapM` — это `traverse`, ограниченный `Monad`; документация `Data.Traversable` прямо говорит: `mapM is literally a traverse with a type signature restricted to Monad`. ²⁷
- `sequenceA` / `sequence` превращают «структуру эффектов» в «эффект структуры». ⁴⁷

Сравнительная таблица (самая полезная для практики):

Функция	Тип (упрощённо)	Когда использовать	Возвращает
<code>map</code>	<code>(a -> b) -> [a] -> [b]</code>	когда функция чистая	список результатов
<code>traverse</code>	<code>(a -> f b) -> t a -> f (t b)</code>	когда функция возвращает эффект (<code>Maybe</code> , <code>Either</code> , <code>IO</code> , <code>State</code> ...) и вы хотите «всё собрать»	один эффект со структурой результатов
<code>mapM</code>	<code>(a -> m b) -> t a -> m (t b)</code>	то же, но вы точно в <code>Monad</code>	то же
<code>mapM_</code> / <code>traverse_</code>	<code>(a -> m b) -> t a -> m ()</code>	когда важны только эффекты , а результаты не нужны	<code>m ()</code>
<code>sequence</code>	<code>t (m a) -> m (t a)</code>	когда у вас уже есть «контейнер действий»	<code>m (t a)</code>

Определения `mapM` / `mapM_` через `sequence` в Prelude тоже дают хорошую «шпаргалку»: `mapM f as = sequence (map f as)`. ⁴⁸

pure и return

- `pure` — метод класса `Applicative`: «поднять значение в контекст». ⁴⁹
- `return` — метод класса `Monad`. В современных библиотеках ожидается совместимость: если тип является и `Applicative`, и `Monad`, то корректно `pure = return`. ²⁴

Практическое правило:

- пишете `Applicative`-стиль (`<$>`, `<*>`, `traverse`) → используйте `pure`;
- пишете `do`-блок → можете использовать `pure` или `return`, но во многих курсах рекомендуют `pure`, потому что он более общий. ⁵⁰

zipWith, lines, words, unlines

- `zipWith` объединяет два списка поэлементно функцией. ⁵¹
- `lines` и `words` — стандартные разбиения строки; `unlines` — обратная операция, склеивающая строки с `\n`. Это отражено и в `Prelude/Report`. ²²

Пример, полезный для задач «обработать ввод построчно»:

```
process :: String -> String
process input =
    unlines (map (unwords . reverse . words) (lines input))
```

Пояснение: 1) `lines input :: [String]` разбивает на строки. ²²

2) Для каждой строки: `words` → список слов; `reverse` → перевернули; `unwords` → склеили словами через пробел. ²²

3) `unlines` склеивает строки обратно с переводами строки. ²²

foldl, foldl', foldr и сложность из-за ленивости

В лабораторных почти всегда проверяют две вещи: 1) умеете ли вы выразить вычисление как свёртку; 2) понимаете ли вы, почему `foldl` иногда «ест память».

Документация `Data.List` формулирует это очень явно:

- `foldl` — **левая** свёртка, *lazy in the accumulator* (ленива по аккумулятору), «редко то, что вы хотите»; для списков обычно лучше `foldl'` или `foldr`. ⁵²
- проблема `foldl`: он строит цепочку «thunk chain O(n)», если оператор не форсирует промежуточные результаты. ⁵³
- `foldl'` — строгая версия, форсит промежуточный аккумулятор до WHNF на каждом шаге и избегает накопления thunks. ⁵³
- `foldr` — правая свёртка, тоже *lazy in the accumulator*, и при функции, ленивой по второму аргументу или коротко-замыкающей, может работать даже с бесконечными структурами. ⁵⁴

Таблица: `foldl` vs `foldr` (и почему часто нужен `foldl'`)

Свёртка	Направление	Ленива/строга	Ключевой практический эффект
<code>foldl</code>	слева направо	ленива по аккумулятору	риск «цепочки thunks» и памяти $O(n)$; бесконечные списки не завершаются ⁵³
<code>foldl'</code>	слева направо	строгая по аккумулятору	почти всегда правильный выбор для сумм/произведений и т.п. ⁵³
<code>foldr</code>	справа налево	ленива по аккумулятору	может выдавать результат на бесконечном списке при коротком замыкании/ленивости по 2-му аргументу ⁵⁴

Пример “почему `foldl` может «висеть»” (из документации):

```
foldl (\a _ -> a) 0 (repeat 1) -- hangs
```

Объяснение: `foldl` должен пройти весь список, чтобы построить «внешнюю» форму выражения; на бесконечном списке это не закончится. ⁵³

Пример “почему `foldr` иногда спасает на бесконечном”:

```
foldr (||) False (True : repeat False) -- True
```

`(||)` коротко замыкается на `True`, и `foldr` может завершиться, потому что голова результата зависит от первых элементов слева. ⁵⁴

`error`, `undefined` и почему их надо «держат под контролем»

`error` и `undefined` — это «частичные» функции/значения: они существуют для «прерванного» вычисления, но ломают безопасность. В стандартной библиотеке:
`error :: HasCallStack => String -> a` бросает исключение с сообщением. ⁵⁵
`undefined :: HasCallStack => a` определён как `error "Prelude.undefined"`. ⁵⁶

Практическое правило для лабораторных: `error/undefined` допустимы как временная заглушка, чтобы компилировалось, но в финальной версии лучше заменить на `Maybe/Either`, или на `die` /исключения в `IO` (если задание про ввод-вывод). ⁵⁷

Операторы, композиция, локальные определения и индексирование

Композиция `(.)` и применение `($)`

В Prelude (Report) даны определения и приоритеты:

- `(.)` — композиция функций: `f . g = \x -> f (g x)`, `infixr 9 .` (высокий приоритет). ⁵⁸
- `($)` — применение с минимальным приоритетом: `f $ x = f x`, `infixr 0 $`. ⁵⁹

Таблица `$` vs `.` vs скобки:

Конструкция	Что делает	Главный эффект в коде	Пример
Скобки (...)	задают порядок вычислений	«явно и понятно», но много шума	<code>print (sum (map (+1) xs))</code>
(\$)	просто применение	убирает лишние скобки справа	<code>print \$ sum \$ map (+1) xs</code> ⁵⁹
(.)	композиция	склеивает функции в конвейер	<code>print . sum . map (+1) \$ xs</code> ⁵⁸

`++`, `show`, `print`, `putStrLn`

- `++` — конкатенация списков (`String` — это `[Char]`), поэтому «склейка строк» — это `+`. Определение `++` есть в `PreludeList (Report)`. ⁶⁰
- `show :: Show a => a -> String` задан как метод класса `Show`; `showsPrec` по умолчанию реализуется через `show` и `++`. ¹⁸
- `print` и `putStrLn` определены в `PreludeIO`: `print x = putStrLn (show x)`, а `putStrLn` реализован через `do`. ¹⁸

Мини-пример «правильной связки»:

```
main :: IO ()
main = do
  let xs = [1,2,3]
  putStrLn ("xs = " ++ show xs)
  print (sum xs)
```

Пояснение: 1) `"xs = " ++ show xs` строит `String`. ⁶¹

2) `putStrLn` печатает строку. ¹⁸

3) `print` сам вызывает `show`. ¹⁸

`let ... in` и `where`, области видимости

`let ... in ...` — выражение, вводит лексически ограниченные определения. В отчёте подчёркнуто: `let` задаёт вложенный, лексически-ограниченный, взаимно рекурсивный список объявлений (аналог `letrec`), и область видимости включает и тело `in`, и правые части объявлений. Также важный нюанс: pattern bindings в `let` матчатся **лениво** (не сразу), «как будто есть неявный `~`». ¹⁴

`where` — часть определения функции/альтернативы `case`: грамматика указывает, что `rhs` может иметь `[where decls]`, и по аналогии с `case`-альтернативами `where`-объявления доступны внутри `guards` и тела соответствующей альтернативы/уравнения. ⁶²

Пара одинаковых примеров:

```

hyp1 :: Floating a => a -> a -> a
hyp1 a b =
    let sq x = x * x
    in sqrt (sq a + sq b)

hyp2 :: Floating a => a -> a -> a
hyp2 a b = sqrt (sq a + sq b)
    where
        sq x = x * x

```

Практический выбор: - `let ... in` — хорошо, когда вы внутри выражения и хотите «маленькую локальную подфункцию». ¹⁴

- `where` — хорошо, когда вы хотите вынести «определения-помощники» рядом с уравнением и использовать их в guards/внутри нескольких строк одного определения. ⁶²

Нюанс “ленивого let” (важно для лабораторных про $\perp$ /undefined):

```

demo :: Int
demo = let (x,y) = undefined in 1

```

По стандарту `let (x,y) = undefined in e` не падает, пока вы не потребовали `x` или `y`. Это типичный «подводный камень» ленивых pattern bindings. ⁶³

Индексация `!!`, безопасный `!?` и матрицы

`!!` — частичная функция (может упасть), индексирование списка с 0. В базе у неё есть ограничения и `HasCallStack`. ⁶⁴

`(!?)` — безопасная альтернатива: возвращает `Nothing`, если индекс вне диапазона, и прямо объявлена как «total variant of partial (!!)». Документация предупреждает: операция линейная по индексу. ⁶⁴

Мини-пример безопасного доступа:

```

import Data.List ((!?))

safeHead :: [a] -> Maybe a
safeHead xs = xs !? 0

```

Доступ к матрице (список списков): типичный «двухшаговый» `Maybe`:

```

import Data.List ((!?))

at2D :: [[a]] -> Int -> Int -> Maybe a
at2D m i j = do

```

```
row <- m !? i
row !? j
```

- Пояснение: 1) `m !? i` возвращает `Maybe [a]`. ⁶⁴
2) Если строки нет — всё станет `Nothing`.
3) Иначе берём `row !? j`.
4) Такой стиль очень хорошо тренирует «композицию ошибок» через `Maybe` и `do`. ⁶⁵

Рекомендация по эффективности для лабораторных:

Если вы часто индексируете по `i`, списки будут давать $O(i)$ и постепенно ухудшать производительность. Документация прямо подчёркивает линейность по индексу для `(!?)`. Для «настоящих матриц» обычно берут массивы/векторы, но это уже зависит от курса и разрешённых библиотек. ⁶⁶

Практические советы, отладка и что ещё полезно повторить

ГHCi: минимальный набор для лабораторных

Документация GHC подчёркивает, что GHCi позволяет интерактивно вычислять выражения, интерпретировать программу и работать с загруженными модулями; подсказку по командам даёт `:?`. ⁶⁷

- Мини-набор команд: - `:load` (или `:l`) — загрузить модуль; `:reload` — пересчитать изменения. ⁶⁷
- `:type (:t)` — вывести тип выражения; документация показывает примеры работы `:type`. ⁶⁸
- `:info (:i)` — посмотреть информацию о типе/классе/функции и инстансы. ⁶⁹
- `:{ / :}` — многострочный ввод. ⁶⁷
- `:set ...` — включать флаги компилятора в GHCi; в документации есть пример `:set -Wmissing-signatures` и пояснение, что `:set` принимает и GHCi-опции (`+...`) и «обычные» опции (`-...`). ⁷⁰

Типичные ошибки студентов и как их предотвращать

Падения из-за частичных функций.

`read`, `!!`, `head`, `foldr1`, `foldl1`, `error`, `undefined` — источники runtime-падений. Документация прямо предупреждает: `read` падает с `error` и “discouraged”; `foldr1` / `foldl1` неполны для пустых структур; `!!` частичен; `error/undefined` всегда потенциально «бомба». ⁷¹

- Практическая замена: - `readMaybe` / `readEither` вместо `read`. ³
- `!?` вместо `!!` (или `Maybe`-логика через pattern matching). ⁶⁴
- `Maybe` / `Either` вместо `error`, если ошибка — нормальная часть логики. ⁷²
- `die` для «фатальной ошибки в IO-программе» (например, неправильные аргументы командной строки): вывод в `stderr` + `exitFailure`. ⁴

Неисчерпывающее сопоставление (non-exhaustive patterns).

Риск возникает в pattern matching и `case`, если не покрыли все варианты; включайте

предупреждения через `:set ...` в GHCi (пример с `-Wmissing-signatures` в документации иллюстрирует механику). ⁷³

Путаница `map` vs `traverse`.

Проверяйте тип:

- хотите `[Maybe a]` → это `map`;

- хотите `Maybe [a]` → это `traverse`. ⁷⁴

QuickCheck и проверка свойств

QuickCheck — стандартный инструмент для property-based testing (в учебных задачах часто проверяют свойства списков/деревьев). Документация/мануал описывает стиль проверки свойств через генерацию тестов. ⁷⁵

Мини-пример для закрепления (списочная алгебра):

```
-- Для QuickCheck обычно пишут:  
-- prop_reverseInvolutive xs = reverse (reverse xs) == xs
```

Идея для лабораторных: 1) выберите инвариант (например, “двойной reverse возвращает исходное”);

2) запишите его как функцию `prop_...`;

3) прогоните QuickCheck — он найдёт контрпример, если вы ошиблись в реализации. ⁷⁶

Что ещё полезно повторить сверх списка

Чтобы легче проходить практику (и чтобы код в лабораторных получался «идиоматичным»), обычно полезно отдельно повторить:

- **Typeclasses и их роли:** `Eq`, `Ord`, `Show`, `Read`, `Functor`, `Applicative`, `Monad`, `Foldable`, `Traversable` (вы уже частично их трогаете через `map`, `traverse`, `pure/return`). ⁷⁷
- **Строгость/ленивость и WHNF:** почему `foldl` строит thunks и почему `foldl'` часто спасает. ⁷⁸
- **Структура модулей и импортов:** где искать функции (Hackage/Haddock), как читать сигнатуры. (Практический минимум — уметь быстро находить `Text.Read`, `Data.List`, `Data.Traversable`, `System.Exit`.) ⁷⁹
- **Стиль:** короткие чистые функции, явные типы, минимум частичных функций, ясные имена, `case` /guards где уместно; в GHCi — пользоваться `:type` / `:info` как «первой линией диагностики». ⁸⁰

Русскоязычные источники для повторения (как дополнение к официальным): - русское издание/перевод “Изучай Haskell во имя добра!” (переводный учебник) ⁸¹

- “Учебник по Haskell” (русский PDF-учебник) ⁸²

¹ ² ¹² ¹³ ¹⁴ ¹⁶ ¹⁷ ³⁷ ⁶³ <https://www.haskell.org/onlinereport/haskell2010/haskellch3.html>
<https://www.haskell.org/onlinereport/haskell2010/haskellch3.html>

3 10 42 45 71 79 Text.Read

<https://hackage.haskell.org/package/base/docs/Text-Read.html>

4 <https://hackage.haskell.org/package/base/docs/System-Exit.html>

<https://hackage.haskell.org/package/base/docs/System-Exit.html>

5 27 44 47 74 <https://hackage.haskell.org/package/base/docs/Data-Traversable.html>

<https://hackage.haskell.org/package/base/docs/Data-Traversable.html>

6 7 9 18 19 20 22 29 30 31 32 38 41 43 46 48 51 58 59 60 61 77 [https://www.haskell.org/
onlinereport/haskell2010/haskellch9.html](https://www.haskell.org/onlinereport/haskell2010/haskellch9.html)

<https://www.haskell.org/onlinereport/haskell2010/haskellch9.html>

8 15 34 35 36 62 <https://www.haskell.org/onlinereport/haskell2010/haskellch4.html>

<https://www.haskell.org/onlinereport/haskell2010/haskellch4.html>

¹¹ <https://www.haskell.org/onlinereport/haskell2010/>

<https://www.haskell.org/onlinereport/haskell2010/>

21 23 33 39 57 65 72 <https://hackage.haskell.org/package/base/docs/Data-Maybe.html>

<https://hackage.haskell.org/package/base/docs/Data-Maybe.html>

24 49 50 <https://hackage.haskell.org/package/base/docs/Control-Applicative.html>

<https://hackage.haskell.org/package/base/docs/Control-Applicative.html>

²⁵ <https://hackage.haskell.org/package/transformers/docs/Control-Monad-Trans-State-Lazy.html>

<https://hackage.haskell.org/package/transformers/docs/Control-Monad-Trans-State-Lazy.html>

26 28 <https://hackage.haskell.org/package/mtl/docs/Control-Monad-State-Lazy.html>

<https://hackage.haskell.org/package/mtl/docs/Control-Monad-State-Lazy.html>

40 <https://hackage.haskell.org/package/base/docs/Data-Either.html>

<https://hackage.haskell.org/package/base/docs/Data-Either.html>

52 53 54 64 66 78 Data.List

<https://hackage.haskell.org/package/base/docs/Data-List.html>

55 56 <https://hackage.haskell.org/package/base/docs/GHC-Err.html>

<https://hackage.haskell.org/package/base/docs/GHC-Err.html>

67 68 69 70 73 80 https://downloads.haskell.org/ghc/latest/docs/users_guide/ghci.html

https://downloads.haskell.org/ghc/latest/docs/users_guide/ghci.html

75 76 <https://www.cse.chalmers.se/~rjmh/QuickCheck/manual.html>

<https://www.cse.chalmers.se/~rjmh/QuickCheck/manual.html>

81 <https://library.tsilikin.ru/>

%D0%A2%D0%B5%D1%85%D0%BD%D0%B8%D0%BA%D0%B0/%D0%9F%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%
Haskell%20and%20functional%20programming/

%D0%98%D0%B7%D1%83%D1%87%D0%B0%D0%B9%20Haskell%20%D0%B2%D0%BE%20%D0%B8%D0%BC%D1%8F%20%

<https://library.tsilikin.ru/%D0%A2%D0%B5%D1%85%D0%BD%D0%B8%D0%BA%D0%B0/%D0%9F%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%>

Haskell%20and%20functional%20programming/%D0%98%D0%B7%D1%83%D1%87%D0%B0%D0%B9%20Haskell%20%D0%B2%D0%BE%20%D0%B

⁸² <https://anton-k.github.io/ru-haskell-book/files/ru-haskell-book.pdf>

<https://anton-k.github.io/ru-haskell-book/files/ru-haskell-book.pdf>